Cpu scheduling

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- $Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)$
- $Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)$

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int n, i, completed = 0, time = 0, min_index;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int AT[n], BT[n], CT[n], TAT[n], WT[n], done[n];

    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i + 1);
        scanf("%d %d", &AT[i], &BT[i]);
        done[i] = 0;
    }

    while(completed < n) {
        min_index = -1;
```

Test Cases 2 | Run Sample Cases | Run All Cases

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer denotes the Arrival Time (AT) of the process and the second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

4

0 6

0 8

```c
        scanf("%d %d", &AT[i], &BT[i]);
        done[i] = 0;
    }

    while(completed < n) {
        min_index = -1;

        for(i = 0; i < n; i++) {
            if(!done[i] && AT[i] <= time) {
                if(min_index == -1 || BT[i] < BT[min_index])
                    min_index = i;
            }
        }


        if(min_index == -1) {
            time++;
        } else {
            time += BT[min_index];
```

Test Cases 2 | Run Sample Cases | Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

Cpu scheduling

Sample Output 1

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00

Average Waiting Time = 7.00

Example 2

Sample Input 2

3

```
        }


        if(min_index == -1) {
            time++;
        } else {
            time += BT[min_index];
            CT[min_index] = time;
            done[min_index] = 1;
            completed++;
        }
    }

    float totalTAT = 0, totalWT = 0;

    for(i = 0; i < n; i++) {
```

Test Cases 2 | Run Sample Cases | Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Example 2

Sample Input 2

3

0 3

5 2

5 1

Sample Output 2

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time = 2.33

Average Waiting Time = 0.33

Select Language : C (GCC 9.2.0)

```
            completed++;
        }
    }

    float totalTAT = 0, totalWT = 0;

    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];
        totalTAT += TAT[i];
        totalWT += WT[i];
    }

    printf("\nAverage Turnaround Time = %.2f\n", totalTAT / n);
    printf("Average Waiting Time = %.2f\n", totalWT / n);

    return 0;
}
```

Test Cases 2 | Run Sample Cases | Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1